Interaktive Visuelle Datenanalyse 2
Praktikum 1: Überwachtes & unüberwachtes Lernen

Termine

Ausgabe	Abgabe	Abnahme (Testat)
06.11.2026	22.11.2026	ab 23.11.2026

Überblick

Ihr trainiert ein kleines CNN auf Fashion-MNIST, nutzt dessen vorletzte Schicht als Embedding, reduziert dieses Embedding für die Darstellung und clustert es anschließend.

Nach dem Praktikum könnt ihr:

- ein CNN trainieren und anhand von Lernkurven, Confusion Matrix und F1 je Klasse beurteilen
- t-SNE und UMAP sinnvoll parametrisieren und ihre Ergebnisse mit Qualitätsmaßen bewerten
- erklären, warum k-Means, DBSCAN und HDBSCAN auf denselben Daten unterschiedliche Cluster liefern

Daten & Setup

Fashion-MNIST (`torchvision.datasets.FashionMNIST`): 70.000 Graustufenbilder von Kleidungsstücken, 28×28 Pixel, 10 balancierte Klassen.

Beispiel: https://github.com/zalandoresearch/fashion-mnist

Split	Größe	Verwendung
Train	50.000	Gewichte lernen
Validation	10.000 (aus dem offiziellen Train-Split, 1.000 je Klasse)	Early Stopping, Lernraten-Anpassung
Test	10.000 (offizieller Test-Split)	einmalige Endbewertung, Embeddings für Aufgabe 2 und 3

Bereitgestellt: `praktikum1_starter.py` lädt die Daten, legt Split und Test-Stichprobe fest und enthält das vorgegebene Modell sowie Hilfsfunktionen für Training (`make_run` , `run_epoch`) und Embedding-Extraktion (`extract`). Daten, Split, Seed, Modell und Test-Stichprobe bleiben fest, damit die Ergebnisse vergleichbar sind. Alles läuft mit einem Seed (0) für PyTorch, NumPy, t-SNE, UMAP und k-Means. `make_run` setzt den Seed für jeden Trainingslauf neu, sodass jeder Lauf mit derselben Initialisierung und Batch-Reihenfolge startet. Auf GPU/MPS sind die Ergebnisse dennoch nicht bitgenau reproduzierbar, aber vergleichbar.

Umgebung einrichten

Benötigt: Python ≥ 3.12. Im Projektordner (mit `requirements.txt`):

```
python3.12 --version              # Windows: py -3.12 --version
python3.12 -m venv .venv          # Windows: py -3.12 -m venv .venv
source .venv/bin/activate         # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python -c "import torch, sklearn; print(torch.__version__, sklearn.__version__)"
```

Alternative mit Anaconda/Miniconda (z. B. wenn kein Python ≥ 3.12 installiert ist; `python3 --version` zeigt eine ältere Version):

```
conda create -n ivda2 python=3.12 -y
conda activate ivda2
python --version
pip install -r requirements.txt
```

Hinweise:

- Unter Linux installiert `torch` von PyPI die CUDA-Version (mehrere GB). Ohne NVIDIA-GPU vorher die CPU-Variante installieren: `pip install torch==2.14.1 torchvision==0.29.1 --index-url https://download.pytorch.org/whl/cpu`, danach `pip install -r requirements.txt`. macOS (MPS) und Windows brauchen das nicht
- Weitere Hilfspakete (z. B. `pandas`, `seaborn`, `tqdm`) dürft ihr verwenden. Tragt sie mit fester Version in die `requirements.txt` ein (`paket==x.y.z`, Version mit `pip show paket` prüfen) und gebt die Datei mit ab
- Die Abgabe muss in einer frischen Umgebung mit `python <skript>.py` laufen, ohne Notebook und ohne absolute Pfade. Daten werden über `get_data()` nach `./data` geladen

Aufgabe 1: Ein kleines CNN trainieren

Trainiert das vorgegebene CNN und bestimmt mit Early Stopping einen guten Stopppunkt.

Vorgegebenes Modell (in `praktikum1_starter.py` implementiert)

Schicht	Ausgabe
Eingabe	28×28×1
Conv 3×3, 16 Filter → BatchNorm → ReLU → MaxPool 2×2	14×14×16
Conv 3×3, 32 Filter → BatchNorm → ReLU → MaxPool 2×2	7×7×32
Conv 3×3, 64 Filter → BatchNorm → ReLU → MaxPool 2×2	3×3×64
Flatten	576
FC → ReLU (Embedding)	64
Dropout (p = 0,3)	64
FC (Logits)	10

Das Modell gibt Logits aus; `F.cross_entropy` wendet die Softmax intern an. Als Embedding dient die Ausgabe der ReLU nach FC 576→64, vor dem Dropout, extrahiert im eval-Modus (`extract`). PyTorch nutzt Inverted Dropout: Skaliert wird bereits im Training, daher muss beim Testen bzw. bei der Inferenz nicht mit (1−p) skaliert werden.

Standard-Training

- Cross-Entropy (`F.cross_entropy`), `torch.optim.AdamW` mit Lernrate 10^{-3} und Weight Decay λ = 0,01 auf alle Parameter (inkl. Bias und BatchNorm), Dropout p = 0,3
- Lernrate · 0,3, wenn sich der Validierungs-Loss 3 Epochen in Folge nicht verbessert (`ReduceLROnPlateau` mit `factor=0.3` und `patience=2` ; `scheduler.step(val_loss)` einmal pro Epoche)
- höchstens 50 Epochen, **Early Stopping** auf dem Validierungs-Loss mit Patience 6: Abbruch, wenn der Validierungs-Loss 6 Epochen in Folge kein neues Minimum erreicht. Den Zustand mit dem niedrigsten Validierungs-Loss speichert ihr bei jeder Verbesserung in eine eigene Datei (`torch.save(model.state_dict(), pfad)`) und ladet ihn vor der Testauswertung zurück (`model.load_state_dict(torch.load(pfad, map_location=device))`). `best_model.pt` (siehe Abgabe) schreibt ihr erst nach dem Training, mit vollständiger Historie
- Train-Loss und -Accuracy sind die Mittelwerte über alle Bilder einer Epoche im Train-Modus (Dropout aktiv, so liefert es `run_epoch`); Validierungswerte werden im eval-Modus berechnet
- Batchgröße 128 im Training (`make_run`). Validierung und `extract` laufen mit Batchgröße 512. Im eval-Modus nutzt BatchNorm die gespeicherten Statistiken, Dropout ist aus und es werden keine Gradienten berechnet; das Ergebnis hängt dort nicht von der Batchgröße ab, größere Batches sind nur schneller
- das finale Modell wird erst ganz am Ende **einmal** auf dem Test-Split ausgewertet und danach nicht mehr verändert. Aufgabe 2 und 3 nutzen die Embeddings genau dieses Modells

Teilaufgaben

Abbildungen könnt ihr z. B. mit `matplotlib` erstellen (in der `requirements.txt` enthalten; für die Confusion Matrix ist `ConfusionMatrixDisplay` erlaubt). Speichert jede Abbildung als PNG (`plt.savefig`) mit beschrifteten Achsen und Legende.

1. Trainiert das vorgegebene CNN. Plottet Loss und Accuracy für Training und Validierung über die Epochen und markiert die Epoche des gespeicherten Modells. Wo beginnt Overfitting, woran erkennt ihr es? Messt

außerdem die Laufzeit je Epoche (Training + Validierung) und die Gesamtlaufzeit des Trainings (`time.perf_counter`), speichert beide.

2. Gebt Accuracy (`sklearn.metrics.accuracy_score`) und den F1-Wert je Klasse auf dem Test-Split an (`sklearn.metrics.f1_score` mit `average=None`). Welche Klasse ist am schwächsten?
3. Plottet die **Confusion Matrix** (zeilennormiert, `sklearn.metrics.confusion_matrix`). Welche Klasse wird am schlechtesten erkannt (Diagonale), und an welche Klassen gehen ihre Fehler? Ist die Verwechslung symmetrisch (vergleicht i→j mit j→i)? Zeigt für die drei häufigsten Paare je zwei bis drei fehlklassifizierte Beispielbilder (`show_images(…)`). Würdet ihr diese Bilder selbst richtig zuordnen? Liegt der Fehler beim Modell oder ist das Bild bzw. Label mehrdeutig?

Aufgabe 2: Dimensionsreduktion

Projiziert die Embeddings mit t-SNE und UMAP auf 2D, bewertet die Layouts und wählt **vor dem Clustering** je ein t-SNE- und ein UMAP-Layout, in denen ihr in Aufgabe 3 alle Clustering-Ergebnisse darstellt.

Vorgehen

- Arbeitet mit dem finalen Modell aus Aufgabe 1 weiter. Extrahiert die 64-dimensionalen Embeddings der vorletzten Schicht für die feste Test-Stichprobe von 5.000 Bildern: `sub = Subset(test_set, test_subsample())` , dann `y, pred, E = extract(model, sub)` . `E` wird nicht standardisiert, Distanzen sind euklidisch. Layouts, `y` und `pred` haben dieselbe Reihenfolge wie `sub`
- **t-SNE** (`sklearn.manifold.TSNE` , `init="pca"` , `random_state=0`) und **UMAP** (`umap.UMAP` , `init="spectral"` , `random_state=0`) laufen direkt auf dem 64D-Embedding. Alle nicht in den Aufgaben genannten Parameter bleiben beim Default (t-SNE: `max_iter=1000` , `learning_rate="auto"` , Barnes-Hut; UMAP: `min_dist=0.1` , `metric="euclidean"`). Mit `random_state` läuft UMAP auf einem Kern; die Warnung dazu ist erwartet
- **Ausrichtung:** Richtet jedes Layout mit `align()` aus dem Starter auf die 2D-PCA des Embeddings aus (`PCA(n_components=2).fit_transform(E)` mit `sklearn.decomposition.PCA` ; intern `scipy.linalg.orthogonal_procrustes`). Die Ausrichtung dient nur dem visuellen Vergleich: Drehung und Spiegelung ändern Trustworthiness und Continuity nicht
- **Qualitätsmaße** für ein 2D-Layout `Y` gegen das 64D-Embedding `E`
  - Trustworthiness: `sklearn.manifold.trustworthiness(E, Y, n_neighbors=K)` ; bestraft Nachbarn im Layout, die im 64D-Raum keine sind (falsche Nachbarn)
  - Continuity: dieselbe Funktion mit vertauschten Räumen, `trustworthiness(Y, E, n_neighbors=K)` ; bestraft 64D-Nachbarn, die im Layout fehlen (verlorene Nachbarn)

Teilaufgaben

1. **Parameterstudie:** t-SNE mit `perplexity` 5, 30 und 100; UMAP mit `n_neighbors` 5, 15 und 100. Färbt alle sechs Layouts nach den wahren Klassen ein (2×3-Raster, eine Zeile je Verfahren, Farbe je Klasse, z. B. Colormap `tab10` , Legende mit den Klassennamen aus `CLASSES`).
2. **Bewertung:** Berechnet für alle sechs Layouts Trustworthiness und Continuity für K = 5, 15, 30, 100, 500 und plottet T(K) und C(K) über K (zwei Diagramme mit je sechs Kurven, K-Achse logarithmisch).
3. **Auswahl:** Wählt je ein t-SNE- und ein UMAP-Layout und begründet eure Wahl anhand der Kurven aus Teilaufgabe 2 und der Layouts aus Teilaufgabe 1.

4. **Gewählte Layouts:** Färbt beide nach den wahren Klassen ein und hebt die fehlklassifizierten Testbilder hervor. Zeichnet dazu alle Punkte blass (z. B. `alpha=0.15`) und legt die Fehlklassifikationen deckend, etwas größer und mit dünnem schwarzem Rand darüber, jeweils in der Farbe ihrer wahren Klasse.
5. **Interpretation:**
   - `perplexity` bzw. `n_neighbors` legt fest, wie viele Nachbarn beim Berechnen des Layouts berücksichtigt werden; K legt fest, wie viele Nachbarn ihr bei der Bewertung prüft. Was passiert mit der Qualität, wenn K größer ist als der Parameter, und warum?
   - Liegen die Fehlklassifikationen an den Grenzen der Klassengruppen im Layout oder mitten in fremden Klassengruppen?
   - Welche Aussage über die Daten hält über alle sechs Layouts hinweg, welche hängt am Parameter?

Aufgabe 3: Clustering

Clustert das 64D-Embedding mit k-Means, DBSCAN und HDBSCAN und stellt jedes Ergebnis in den beiden in Aufgabe 2 gewählten Layouts nebeneinander dar: einem t-SNE-Layout und einem UMAP-Layout.

Vorgehen

- Alle Verfahren laufen auf dem **64D-Embedding**
- Färbt Cluster-IDs nicht in derselben Farbskala wie die Klassen (z. B. `tab20b` statt `tab10`); Rauschen (Label −1) grau. Da sich die Farben bei mehr als 20 Clustern wiederholen, schreibt die Cluster-ID an den Median jedes Clusters im Layout (`ax.text`)

Teilaufgaben

1. **k-Means:** `sklearn.cluster.KMeans(n_clusters=10, n_init=10, random_state=0)`.
2. **DBSCAN** (`sklearn.cluster.DBSCAN`) mit minPts = 50 (`min_samples=50`). Eine Klasse umfasst in der Stichprobe etwa 500 Punkte; minPts = 50 verlangt also, dass ein Kernpunkt in einer Nachbarschaft von etwa einem Zehntel einer Klasse liegt. Kleinere Ansammlungen bilden keine eigenen Cluster. Bestimmt ε über einen k-Distanz-Plot: `nn = NearestNeighbors(n_neighbors=50).fit(E)` (`sklearn.neighbors.NearestNeighbors`), dann `dist, _ = nn.kneighbors(E)` und `np.sort(dist[:, -1])` plotten. Mit `E` als Argument zählt der Punkt selbst als erster Nachbar mit, so wie bei `min_samples` in DBSCAN; ohne Argument wäre der Plot um einen Nachbarn verschoben. Beschreibt, wie ihr ε bestimmt habt. Zeigt außerdem, was bei ε · 0,5 passiert.
3. **HDBSCAN** (`sklearn.cluster.HDBSCAN`) mit `min_cluster_size=50` und `min_samples=10`, einmal mit `cluster_selection_method="eom"`, einmal mit `"leaf"`. Wie unterscheiden sich Clusterzahl und Rauschanteil? Gebt für jeden Cluster und für das Rauschen an, welche Klassen er mit welchem Anteil enthält (z. B. mit `sklearn.metrics.cluster.contingency_matrix(y, labels)`; Zeilen = Klassen, Spalten = Cluster-IDs aufsteigend, Rauschen −1 zuerst).

   Hinweis: `min_samples` zählt in sklearn den Punkt selbst mit (wie im HDBSCAN-Paper von Campello et al.), das Paket `hdbscan` nicht. Wir clustern mit sklearn; `hdbscan` dient nur zur Berechnung von DBCV.
4. **Bewertung mit DBCV:** Berechnet für alle DBSCAN- und HDBSCAN-Ergebnisse den Density-Based Clustering Validation Index (Moulavi et al., 2014) mit `hdbscan.validity.validity_index(E.astype(np.float64), labels)` aus dem Paket `hdbscan`. DBCV liegt zwischen −1 und 1 (höher = besser) und bewertet Cluster nach

Dichte, ohne die Klassen zu kennen. Jeder Cluster geht mit seinem Anteil an allen Punkten ein; Rauschen senkt den Wert also. Ergebnisse mit nur einem Cluster liefern `nan`, solche ohne Cluster 0.

5. **Interpretation:**
   - Interpretiert den k-Distanz-Plot und die Unterschiede zwischen ε und ε · 0,5
   - Welche Klassen fasst HDBSCAN zusammen, welche trennt es? Wie unterscheiden sich EOM und Leaf darin? Vergleicht mit der Confusion Matrix aus Aufgabe 1
   - Welches Ergebnis hat den höchsten DBCV? Passt es auch am besten zu den Klassen? Warum kann ein Clustering, das mehrere Klassen zusammenfasst, nach DBCV gut abschneiden?
   - Sehen dieselben Cluster in beiden Layouts gleich aus? Was sagt das über die Layouts?

Abgabe

☐ ein Skript, das mit einem einzigen Aufruf (`python <skript>.py`) alle Abbildungen aus Aufgabe 1–3 mit festem Seed erzeugt: nur relative Pfade, Daten werden automatisch nach `./data` geladen (nicht mit abgeben), alle Abbildungen und Kennzahlen landen in einem Ausgabeordner

☐ Kennzahlen (Laufzeiten des Trainings aus Aufgabe 1, Accuracy, F1, T/C-Werte, Clusterzahl, Rauschanteil, Klassenanteile je Cluster, DBCV) als Datei im Ausgabeordner (z. B. JSON oder CSV)

☐ `requirements.txt` mit festen Versionen

☐ `best_model.pt` mit Gewichten (`state_dict`) und Trainingshistorie (Loss/Accuracy und Laufzeit je Epoche, Gesamtlaufzeit, beste Epoche). Mit `python <skript>.py --load best_model.pt` (im Starter angelegt) wird das Training übersprungen; alle Abbildungen aus Aufgabe 1–3 entstehen aus dem geladenen Modell und der gespeicherten Historie.

☐ Antworten auf die Fragen werden bei der Abnahme (Testat) im Gespräch besprochen